

JDT Schema overview

JSON Document Type Schema Language overview. Based on the TOON Schema overview
<https://toonformat.dev/guide/format-overview>.

Data Model

- Primitive: String, number, boolean, and `null`
- Objects and properties (called containers)
- Unstructured arrays
- Defined Data Types

A number can be a floating-point value.

Root

If not specified, the root object exists without explicit declaration. There are certain parameters that the root object can be specified with the keyword `is`.

- `schema` specifies the URI for a parent schema
- `version` specifies the URI of the version used
- `owner` specifies the owner's URL
- `type` specifies the type of the root object

All root parameters are optional. The defaults for root parameters are

- no root schema,
- newest stable version,
- `null`, and
- object

respectively.

Comments are marked by `"""` and are valid until closed with the same sequence.

Valid Names

Names can contain any Unicode character except for `"""` and cannot be `is` or contain a space. Users can use any breakout character for `is`: like `#is`, `\is`, `$is`, It can't be just `is`. For proprietary names, quotation marks `"` can be used. They must be closed and cannot contain other quotation marks.

Example for valid names:

```
name
key
Straße
København
13.03.2004
```

```

new1
wow!?
new_Name
=$%$&@€
isNull"
null
gän"se"füß"chen
#name
"name"
#is
\is
is$
"is"
"name:"

```

Not valid names:

```

is
b""1
n a m e
"na""me"

```

It's the developer's responsibility to ensure their names don't violate JSON's name requirements.

A colon `:` at the end of a name, followed by a return signalizes a container. A key's name ends with the absence of a colon and a space. The indentation `whitespace` encodes the parent.

```

name:           <-- This is a container
name           <-- This is a key
=$%$&@€:       <-- This is a container
=$%$&@€        <-- This is a key

parent:
  child         <-- Indent defines hierarchy

```

Constraints

JDT documents are validated against a JDT Schema with constraints. A constraint starts with `is`.

Simple constraint:

- true
- false
- null
- number
- boolean
- string

Occurrence constraints:

- required: exactly one occurrence
- optional: zero or one occurrence

If not specified, the default is **optional**.

Operator constraints:

- and
- or
- not

Operators follow the standard precedence rules, with **not** having the highest precedence, followed by **and**, and then **or**. Parentheses can be used to explicitly define the order of operations.

Array constraints:

- array
- array(datatype)
- array(datatype, constraint)
- array(array(datatype, constraint), constraint)

Arrays can be nested. If no datatype is specified, all datatypes validate. Constraints can be used inside arrays:

```
hobbies is array(string), required
hobbies is array(string, longer 0), required
hobbies is array(string, longer 0, not "coding"), required
```

That constraints apply directly to the array layer they are written inside.

Value and length constraints:

- minimum [number] for $\geq$
- maximum [number] for $\leq$
- longer [number] for string length or array length $\geq$
- shorter [number] for string length or array length $\leq$
- larger [number] for numeric values $>$
- smaller [number] for numeric values $<$

Equality constraints:

- number
- "string"
- boolean
- null

Match constraints:

- `match("regex")`
- `match([key])`

Regexes and strings must not contain `""`. If a key is not found in the schema, it is considered invalid for a match constraint.

Undefined constraints are not used for a key in the schema, but rather for unknown, potential keys:

- `closed`
- `open`

JDT enforces an open schem. Any key that is not defined in the schema is considered valid unless stated otherwise. Undefined constraints can only be defined once per container at its beginning or for the entire document.

The concardination of constraints is done with a comma `,`:

```
name is string, required, longer 0
```

Defined Data Types

JDT allows custom-defined data types with the keyword `defined`:

```
is defined Address:
  street is string
  number is number, minimum 1
  zip is number, not null

user:
  home is Address
```

The name of the custom data type has to follow the same rules as other names. They must not be the same as an already existing key (case sensitive). A `defined` block is not a container.

Examples

Simple Objects and Constraints:

```
id is number          "" This is a comment ""
name is string
```

Nested Objects and Constraints:

```
user:
  id is number, required, minimum 0
```

name is string, optional

Complex Objects and Constraints:

```
is type object
user:
  is closed
  id is number, required, not null
  name is required, not boolean
  age is optional, (number or null)
  hobbies is array(string), required
  eyecolor is "blue" or "green" or "brown"
```

Realization of the Complex Objects and Constraints Schema:

```
{
  "user": {
    "id": 1,
    "name": "Jane Doe",
    "age": 28,
    "hobbies": [
      "reading",
      "coding",
      "hiking"
    ],
    "eyecolor": "blue"
  }
}
```

<-- This is the root object
 <-- This is a container
 <-- This is a key value pair
 <-- This is an array consisting of strings

This is valid. The following is not:

```
{
  "user": {
    "id": null,
    "name": true,
    "age": "twenty",
    "sex": "female",
    "height": 172
  }
}
```

<-- Violates Schema `not null`
 <-- Violates Schema `not boolean`
 <-- Violates Schema `(number or null)`
 <-- Violates Schema key
 <-- Violates Schema `closed`
 <-- Violates Schema hobbies `required`

Longer Example

```

is schema "https://www.uni-regensburg.de/jdt-schema"
is version "https://www.jdt-schema.com/version/1"
is owner "https://www.uni-regensburg.de"
is type object

is closed

is defined Address:
  street is string
  number is number, minimum 1
  zip is number, not null

is defined Payment:
  number is number, required, longer 13, shorter 19
  name is string, required
  expires is match("""\b(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])\b"""),
required
  cvv is number, required, longer 3, shorter 4

is defined Year:
  year is number, larger 2020, smaller 2030

user:
  id is number, required, minimum 0
  name is string, longer 0, required
  address is Address, required
  sex is string, optional
  height is number
  eyecolor is string, "blue" or "green" or "brown", optional
  creditCard is Payment, required
  subscribed is boolean
  memberSince is Year

```

Validates:

```

{
  "$schema": "https://www.uni-regensburg.de/jdt-schema",
  "$version": "https://www.jdt-schema.com/version/1",
  "$owner": "https://www.uni-regensburg.de",
  "user": {
    "id": 101,
    "name": "Jane Doe",
    "address": {
      "street": "Universitätsstraße",
      "number": 31,
      "zip": 93053
    },
    "sex": "female",
    "height": 172,
    "eyecolor": "green",

```

```
"creditCard": {  
  "number": 4532123456789012,  
  "name": "Jane Doe",  
  "expires": "14/08",  
  "cvv": 987  
},  
"subscribed": true,  
"memberSince": {  
  "year": 2025  
}  
}
```